

K-fold交叉驗證說明

麻煩以:在機器學習領域中, 以K-fold「交叉驗證」(Cross-Validation) 為題目,作一份詳細的說明及舉例
範例假設: 收集資料500筆, K=5

400 訓練

50 驗證

50 測試

-

從:

1.如何: 訓練, 驗證, 選出一組最佳效能(穩定)的超參數 (訓練這5次時的超參數都是用一樣的)
幾組的 Learning Rate * 幾組的 Batch Size * 幾組的 Optimizer

2.如何選出最佳的超參數呢?

A.傳統的 Grid Search (網格搜尋)

B.依文獻固定

C.逐步調整

3.以此組超參數, 去訓練所有的訓練集的資料

3.A)有的要留驗證集(400 訓練)

3.B)有的可以不用留驗證集(450 訓練)

4.從訓練流程 (Training Procedure) 來看, 很多實驗都會留 Validation

因為有些模型在訓練時, 需要一個 Validation Set 來決定:

Early Stopping, Model Checkpoint, 儲存最佳權重, ReduceLROnPlateau (自動降低 Learning Rate) ,
選擇最佳 Epoch

5.如果在第(2)步驟, 一直選不到最佳效能(穩定)的模組

會有什麼好的建議呢?

顯示更多顯示較少

在機器學習領域中, 以 K-fold「交叉驗證 (Cross-Validation)」為題目

以下以一個完整的研究流程來說明 K-fold Cross-Validation 在機器學習與深度學習中的使用方式。

一、為什麼需要 K-fold Cross Validation ?

假設我們收集了 **500 筆資料**。

如果只切一次

- Training : 400
- Validation : 50
- Testing : 50

那麼模型的好壞, 很可能受到「剛好切到哪些資料」影響。

例如：

第一次切法

Training : 400
Validation : 50

Accuracy = 95%

但是如果重新亂數切一次

Training : 400
Validation : 50

Accuracy 可能只剩

91%

表示模型不夠穩定。

因此便發展出 **K-fold Cross Validation**（交叉驗證）。

目的就是：

降低資料切割造成的偶然誤差，使模型評估更可靠。

二、本例資料分配

假設共有

500 筆資料

一般流程會先保留 Test Set。

例如：

500
├─ 450 Training Pool
└─ 50 Testing

這 50 筆 Test

整個研究都不能碰。

直到最後才使用。

接著

450

拿來做

K = 5

所以

每 Fold

```
450 / 5 = 90
```

因此

每一次會變成

```
360 Training  
90 Validation
```

共做五次。

五次交叉驗證如下

Fold1

```
Train :  
2 3 4 5
```

```
Valid :  
1
```

Fold2

```
Train :  
1 3 4 5
```

```
Valid :  
2
```

Fold3

```
Train :  
1 2 4 5
```

```
Valid :  
3
```

Fold4

```
Train :  
1 2 3 5
```

```
Valid :  
4
```

Fold5

```
Train :  
1 2 3 4
```

```
Valid :  
5
```

每一筆資料

都會當一次 Validation。

三、第一步：固定一組超參數

例如：

Learning Rate = 0.001

Batch Size =16

Optimizer = Adam

注意：

這五次 CV 的超參數完全一樣。

不能 Fold1 用 Adam

Fold2 用 SGD

否則比較就沒有意義。

五次訓練

例如

Fold	Validation Accuracy
1	92.1%
2	93.5%
3	91.8%
4	92.7%
5	92.4%

最後得到

Mean

92.5%

Standard Deviation

±0.6%

表示

這組超參數的平均效能

以及穩定性。

四、如何尋找最佳超參數？

真正研究時

通常不是只測一組。

例如：

Learning Rate

```
0.1
0.01
0.001
0.0001
```

Batch Size

```
8
16
32
```

Optimizer

```
SGD
Adam
AdamW
```

總共

```
4 × 3 × 3
=36 組超參數
```

每一組

都必須跑

```
5-fold CV
```

因此

總共需要

```
36 × 5
=180 次模型訓練
```

五、如何選最佳超參數？

主要有三種方式。

A. Grid Search（網格搜尋）

最標準的方法。

例如：

Learning Rate

0.1
0.01
0.001

搭配

Batch Size

8
16
32

搭配

Optimizer

SGD
Adam
AdamW

全部組合都跑。

例如：

LR	Batch	Optimizer	Mean Accuracy	Std
0.01	16	Adam	94.2	± 1.5
0.001	16	Adam	94.0	± 0.3
0.0001	16	Adam	93.8	± 0.2

很多人會選

94.0 ± 0.3

而不是

94.2 ± 1.5

因為

平均略低

但非常穩定。

B. 依文獻固定

例如：

很多 U-Net

都使用

Adam

Learning Rate

0.0001

Batch Size

16

研究者便直接沿用。

這也是目前論文最常見的方法。

原因：

- 文獻已有驗證。
- 可降低實驗成本。
- 提高研究可重現性。

C. 逐步調整 (Sequential Tuning)

很多研究都採這種方式。

例如：

先固定

Batch=16

Optimizer=Adam

只調

Learning Rate

找到最佳

例如

0.001

再固定

Learning Rate=0.001

開始調

Batch Size

例如

8

16

32

找到最佳。

最後

再調

Optimizer。

這種方式

比 Grid Search 快很多。

也是深度學習常見的方法。

六、如何決定最佳超參數？

不能只看 Accuracy。

還要看

Mean
+
Standard Deviation

例如

A

95.1 ±2.8

B

94.9 ±0.2

通常很多研究

反而選 B。

因為：

- 平均只差 0.2%
- 穩定性高很多
- 泛化能力通常較佳

因此，超參數的選擇應兼顧效能（Mean）與穩定性（Standard Deviation），而不是只追求最高的平均分數。

七、找到最佳超參數後怎麼辦？

假設最後得到

Learning Rate=0.001

Batch Size=16

Adam

這就是

最佳超參數。

接著開始

Final Training。

方法 A（大部分深度學習）

仍保留 Validation。

例如

450

↓

400 Training

50 Validation

原因：

需要

- Early Stopping
- ReduceLROnPlateau
- Model Checkpoint
- Best Epoch
- 儲存最佳權重

因此不能沒有 Validation。

最後

再用

50 Test

做一次

真正測試。

方法 B（部分機器學習）

有些演算法

例如：

- 傳統線性模型
- 決策樹
- 隨機森林
- 支援向量機（scikit-learn 中許多經典演算法）

通常沒有

Epoch

也沒有

Early Stopping

因此

可直接使用

450

全部拿去訓練

最後

測試

50

即可。

不過，若這些演算法本身也需要調整超參數（例如 SVM 的 C 、 γ ，Random Forest 的樹數與深度），仍然應在交叉驗證階段完成，而不是利用 Test Set 調整。

八、為什麼深度學習大多需要 Validation ？

因為訓練時

每一個 Epoch

都需要知道

模型是否進步。

例如：

Epoch

10

Accuracy

90%

Epoch

20

92%

Epoch

30

94%

Epoch

40

93%

代表開始 Overfitting。

因此

最佳模型

其實是

Epoch30

而不是

Epoch40

所以 Validation 必不可少。

Validation 主要用途包括：

1. **Early Stopping**：當驗證集表現不再改善時提前停止訓練，避免過度擬合。
2. **Model Checkpoint**：保存驗證集表現最佳時的模型權重。
3. **ReduceLROnPlateau**：當驗證集指標停滯時，自動降低 Learning Rate。
4. **選擇最佳 Epoch**：以驗證集最佳表現對應的 Epoch 作為最終模型，而不是最後一個 Epoch。

九、如果一直找不到最佳且穩定的超參數，該怎麼辦？

若所有超參數組合都呈現：

Accuracy

忽高忽低

CV

±很大

代表

模型穩定性不好。

可依序考慮：

(1) 增加資料量（最有效）

例如

```
500
→1000
→2000
```

通常比一直調超參數更有效。

(2) 增加資料多樣性

例如：

不同醫院

不同設備

不同年齡

不同光照

提升泛化能力。

(3) 改善資料品質

例如：

- 修正錯誤標註
 - 排除品質差影像
 - 統一標註規則
-

(4) 合理的資料增強（Data Augmentation）

例如：

- Rotation
- Flip（需符合任務特性）
- Brightness
- Contrast
- CLAHE
- Gaussian Noise

避免產生不符合實際情境的影像。

(5) 使用 Transfer Learning

例如：

使用 ImageNet 預訓練權重初始化 Encoder，再進行 Fine-Tuning，通常可提升小型資料集的穩定性與收斂速度。

(6) 更換模型架構

例如：

由 U-Net

改成

- U-Net++
- Attention U-Net
- DeepLabV3+
- SegFormer

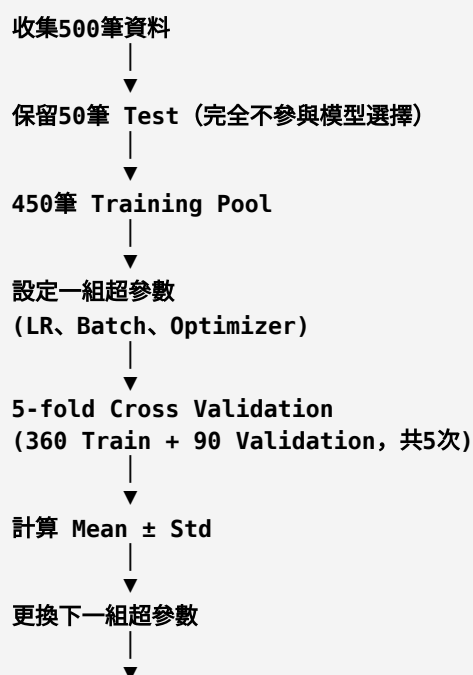
有時改善幅度遠大於微調 Learning Rate。

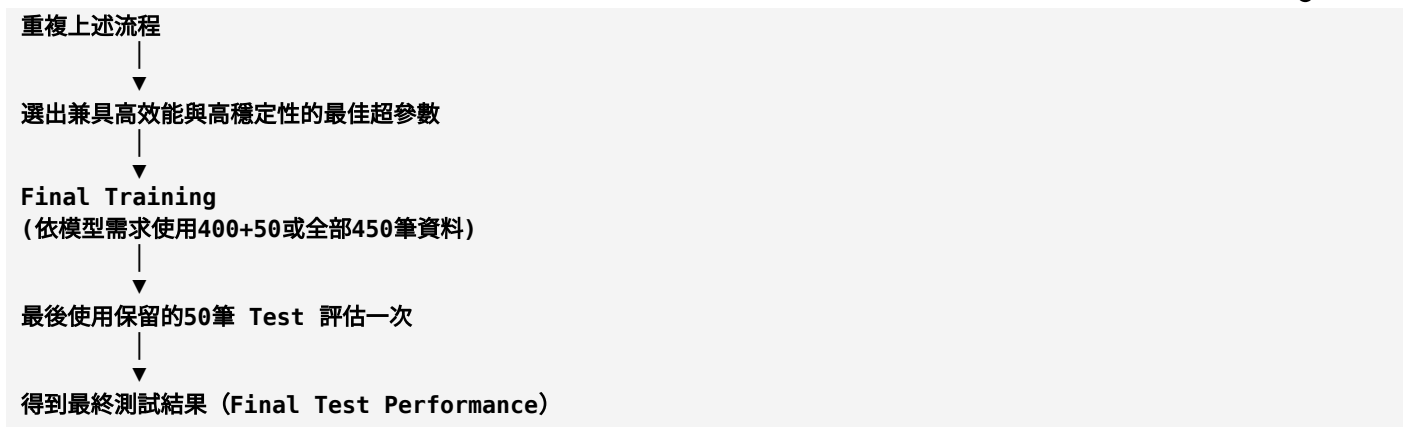
(7) 擴大超參數搜尋方式

除了 Grid Search 外，也可考慮：

- **Random Search**：在超參數空間中隨機抽樣，效率通常優於 Grid Search。
- **Bayesian Optimization**：根據前次實驗結果，智慧化挑選下一組超參數，能以較少次數找到較佳解。

十、完整流程總結





重點觀念

1. **Test Set (50 筆)** 只能在最後使用一次，不可用來挑選超參數或模型。
2. **K-fold CV** 的目的不是產生最後部署的模型，而是評估模型與選擇最佳超參數。
3. 最佳超參數應兼顧平均效能 (Mean) 與穩定性 (Standard Deviation)，而非只追求最高平均值。
4. 深度學習模型通常在 **Final Training** 仍保留 **Validation Set**，以支援 **Early Stopping**、**Model Checkpoint**、**Learning Rate Scheduler** 等訓練機制；部分傳統機器學習演算法則可直接使用全部訓練資料進行最終訓練。
5. 若交叉驗證結果始終不穩定，優先改善資料品質、資料量與模型設計，通常比無止境微調超參數更有效。